

《程序开发实践》项目报告

Project Report on Programming in Applications

图书管理系统

Book Management System

学 院	东北大学悉尼智能科技学院
专 业	计算机科学与技术
班级序号	2502
学 号	202512898
姓 名	田佩宁
指导教师	王和兴
验收日期	2026 年 1 月 10 日

核心知识点清单，由学生确认

数组	链表	指针	文件读写	默认参数	函数模板	多文件	类	派生	虚函数	友元函数	重载	多继承
√	√	√	√	√	√	√	√			√	√	

以下为教师评分表，学生不可填写

程序质量 (60%)	课程设计报 告 (20%)	答辩效果 (20%)	总分	等级

评分标准与说明：

- 1、程序质量（60%）包含程序正确性与所用知识点数量（40%），代码可读性（10%）与界面友好性（10%）。
- 2、课程设计报告（20%）要求排版规范，模块设计有文字说明，图、表、代码清单要有序号和名字。
- 3、现场答辩（20%）要求根据学生制作的 PPT、讲述清晰、回答问题等情况综合评分。
- 4、要求学委收集所有学生的代码工程、报告电子版和答辩 PPT 以备存档。

目录

1	课程设计要求	04
2	项目设计概览	05
2.1	模块结构及其流程	05
2.2	工程组织结构	07
2.3	技术实现	07
2.4	调试过程及运行结果	10
3	得意之处	12
4	亟待完善之处	12
5	收获与体会	13
6	参考文献	14

1 课程设计要求

1、问题描述

定义图书类，属性有：书名、出版社、ISBN 号、作者、库存量、价格等信息和相关的对属性做操作的行为，完成对图书的销售、统计和图书的简单管理。

2、功能要求

（1）图书简单管理功能。

添加功能：主要完成图书信息的添加，要求 ISBN 号唯一。当添加了重复的编号时，则提示数据添加重复并取消添加。

查询功能：可按书名、ISBN 号、作者、出版社进行查询。若存在相应信息，输出所查询的信息，若不存在该记录，则提示“该标题不存在！”。

修改功能：可根据查询结果对相应的记录进行修改，修改时注意 ISBN 号的唯一性。

删除功能：主要完成图书信息的删除。输入要删除的 ISBN 号，根据编号删除该物品的记录，如果该编号不在物品库中，则提示“该编号不存在”。

（2）销售功能。

购买书籍时，输入相应的 ISBN 号，并在书库中查找该书的相关信息。如果有库存量，输入购买的册数，进行相应计算。如果库存量不够，给出提示信息，结束购买。

（3）统计功能。

输出当前书库中所有图书的总数及详细信息；可按书的价格、库存量、作者、出版社进行统计，输出统计信息时，要按从大到小进行排序。

（7）图书存盘：将当前程序中的图书信息存入文件中。

（8）读出信息：从文件中将图书信息读入程序。

2 项目设计概览

2.1 模块结构及其流程

该题目的题面给出了详细的需求，包括增删改查、出售、统计、存取档等功能。可以据此画出该图书管理系统的结构图（如图 1 所示）。

在分析需求的同时，也应当考虑一些细节的实现手段。在这道题目中体现为：

✚ 用什么工具来制作 UI 界面？

Ø FLTK，时间成本低，环境配置简单；

✚ 查询/统计使用的算法？

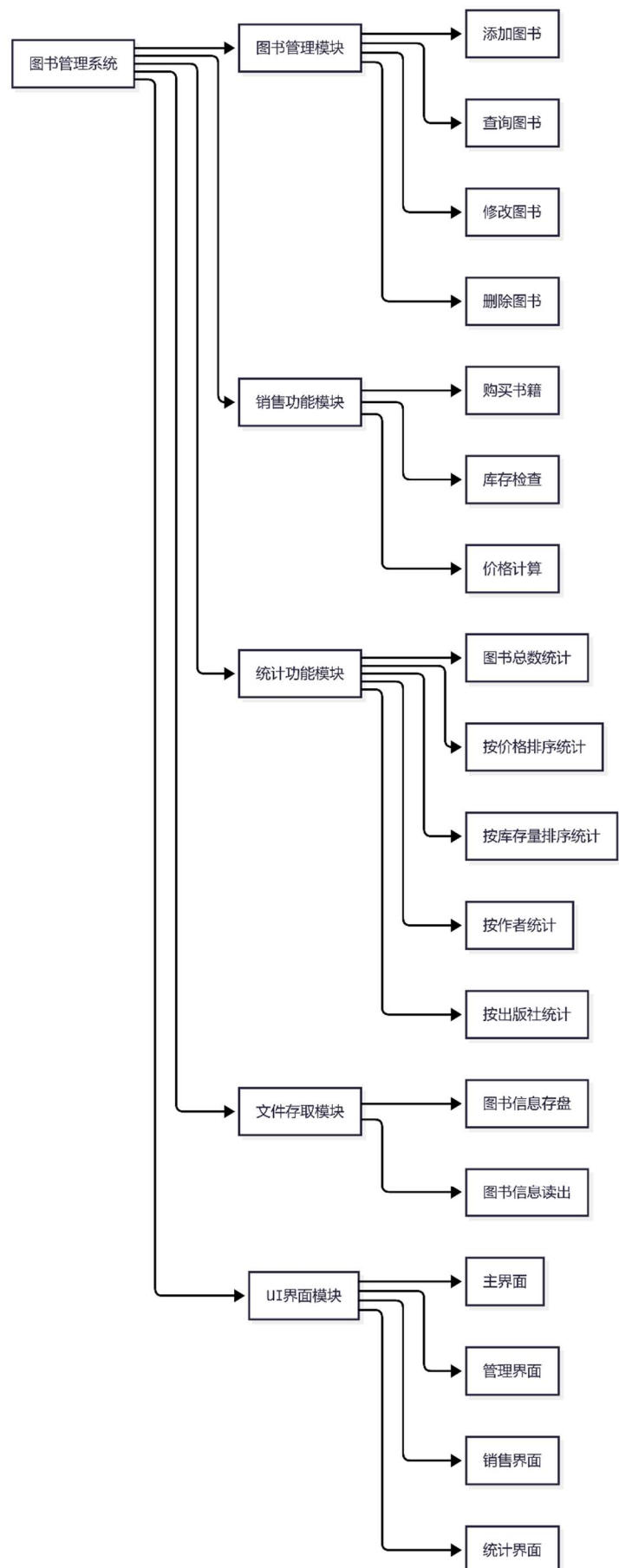
Ø STL 模板即可，因为数据量不大；

✚ ASCII 文件存取还是二进制文件存取？

Ø 二进制，可以锤炼本领(?)

✚ 要采用现代 C++ 工具吗？

Ø 适度使用，std::vector 容器、lambda 表达式在这道题很有用；



▲ 图 1 模块结构图

据此，设计出 5 个基于 has-A 关系的类（如图 2 所示）。

- Book 类

属性：书名、出版社、ISBN 号、作者、库存量、价格

作用：构造函数、getter/setter 方法、字符串转换方法

- BookManager 类

属性：图书容器（vector<Book>）

作用：增删改查、排序、文件存取

- SaleSys 类（Sys 是 System 的缩写）

属性：指向 BookManager 的指针

作用：购买书籍、价格计算

- StatisSys 类（Statis 是 Statistics 的缩写）

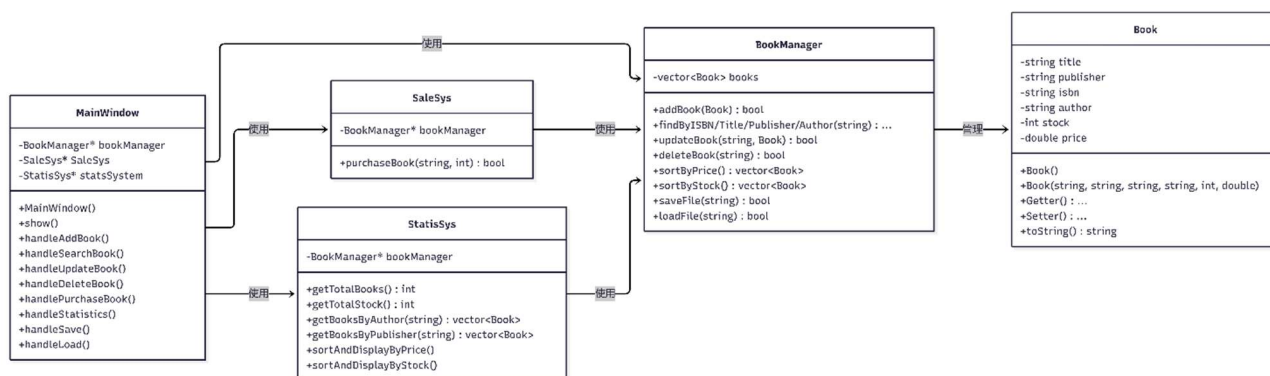
属性：指向 BookManager 的指针

作用：各类统计功能

- MainWindow 类

属性：各系统指针、UI 组件

作用：界面事件处理、功能调用



▲图 2 各个类的实现及其关系

2.2 工程组织结构

```
BMS/
├── include/
│   ├── Book.h
│   ├── BookManager.h
│   ├── SaleSys.h
│   ├── StasisSys.h
│   └── MainWindow.h
├── src/
│   ├── Book.cpp
│   ├── BookManager.cpp
│   ├── SaleSys.cpp
│   ├── StasisSys.cpp
│   ├── MainWindow.cpp
│   └── main.cpp
├── data/
│   └── books.dat
├── docs/
│   └── PIA 课程项目报告.docx
└── CMakeLists.txt
```

2.3 技术实现

```
58  /* STAR: binary read&write */
59  // ==binary write==
60  std::ostream& operator<<(std::ostream& os, const Book& book) {
61      // ISBN(长度 + 字符串)
62      int isbnLen = book.isbn.length();
63      os.write(s: reinterpret_cast<const char*>(&isbnLen), n: sizeof(isbnLen));
64      os.write(s: book.isbn.c_str(), n: isbnLen);
65      // 书名
66      int titleLen = book.title.length();
67      os.write(s: reinterpret_cast<const char*>(&titleLen), n: sizeof(titleLen));
68      os.write(s: book.title.c_str(), n: titleLen);
69      // 作者
70      int authorLen = book.author.length();
71      os.write(s: reinterpret_cast<const char*>(&authorLen), n: sizeof(authorLen));
72      os.write(s: book.author.c_str(), n: authorLen);
73      // 出版社
74      int publisherLen = book.publisher.length();
75      os.write(s: reinterpret_cast<const char*>(&publisherLen), n: sizeof(publisherLen));
76      os.write(s: book.publisher.c_str(), n: publisherLen);
77      // 库存 价格
78      os.write(s: reinterpret_cast<const char*>(&book.stock), n: sizeof(book.stock));
79      os.write(s: reinterpret_cast<const char*>(&book.price), n: sizeof(book.price));
80
81      return os;
82  }
```

▲图 3 利用输入流重载插入符（提取符为其逆过程）

```

26 // 根据ISBN查找图书(Note: 已经确保ISBN具有唯一性)
27 Book* BookManager::findByISBN(const std::string& isbn) {
28     int index = findIndex(isbn);
29     if (index != -1) {return &books[index];}
30     return nullptr;
31 }
32
33 const Book* BookManager::findByISBN(const std::string& isbn) const {
34     int index = findIndex(isbn);
35     if (index != -1) {return &books[index];}
36     return nullptr;
37 }
38
39 // 根据书名查找图书
40 std::vector<Book*> BookManager::findByTitle(const std::string& title) {
41     std::vector<Book*> result;
42     for (auto& book: Book & : books) {
43         if (book.getTitle().find(str: title) != std::string::npos) {
44             result.push_back(x: &book);
45         } // STAR: 模糊查询, 如果有字符串片段即查询成功
46     }
47     return result;
48 }
49
50 std::vector<const Book*> BookManager::findByTitle(const std::string& title) const {
51     std::vector<const Book*> result;
52     for (const auto& book: const Book & : books) {
53         if (book.getTitle().find(str: title) != std::string::npos) {
54             result.push_back(x: &book);
55         }
56     }
57     return result;
58 }

```

▲图 4 部分查找算法的实现过程

```

102 // 更新图书信息
103 bool BookManager::updateBook(const std::string& isbn, const Book& newBook) {
104     int index = findIndex(isbn);
105     if (index == -1) {return false;}
106
107     // 如果ISBN改变, 要检查新的ISBN是否已存在
108     if (newBook.getISBN() != isbn && findIndex(isbn: newBook.getISBN()) != -1) {
109         return false; // 新的ISBN已存在
110     }
111
112     books[index] = newBook;
113     return true;
114 }
115
116 // 删除图书
117 bool BookManager::deleteBook(const std::string& isbn) {
118     int index = findIndex(isbn);
119     if (index == -1) {return false;} // 图书不存在
120
121     books.erase(position: books.begin() + index); // vector库函数
122     return true;
123 }

```

▲图 5 修改与删除功能的实现


```

135 // 按价格排序 (decreasing)
136 std::vector<Book*> BookManager::sortByPrice() {
137     std::vector<Book*> result;
138     for (auto& book: Book & : books) {
139         result.push_back(x: &book);
140     } // 复制一个临时数组
141
142     std::sort(first: result.begin(), last: result.end(),
143         comp: [](Book* a, Book* b)->bool{return a->getPrice() > b->getPrice();}); // STAR: lambda表达式
144
145     return result;
146 }
147
148 // 按库存量排序 (decreasing)
149 std::vector<Book*> BookManager::sortByStock() {
150     std::vector<Book*> result;
151     for (auto& book: Book & : books) {
152         result.push_back(x: &book);
153     }
154
155     std::sort(first: result.begin(), last: result.end(),
156         comp: [](Book* a, Book* b)->bool{return a->getStock() > b->getStock();});
157
158     return result;
159 }

```

▲图 6 在 `std::sort()` 函数中使用 `lambda` 表达式，从而简化逆序排列写法

```

161 // 保存到文件
162 bool BookManager::saveFile(const std::string& filename){
163     std::ofstream file(s: filename, mode: std::ios::binary);
164
165     if (!file) {return false;} // 失败1
166     try {
167         int amount = static_cast<int>(books.size()); // 图书总数
168         file.write(s: reinterpret_cast<const char*>(&amount), n: sizeof(amount));
169
170         // 写入每本图书
171         for (const auto& book: Book const & : books) {
172             file << book;
173         }
174
175         file.close();
176
177         return true;
178     }catch (...) { // ...代指多种类型的异常
179         if (file.is_open()) {
180             file.close();
181         }
182         return false; // 失败2
183     }
184 }

```

▲图 7 利用异常处理解决文件存取中的问题

2.4 调试过程及运行结果

本项目所有代码均在 MSYS2 UCRT64 环境下调试，编译器为 CLANG 21.1.8，经 CMake 构建后生成 BMS.exe 可执行文件。开发过程产生的文件均已上传至 [Github 仓库](#)



▲图 8 主操作界面



▲图 9 执行查询“C++”操作后界面的显示



▲图 10 执行“统计信息”后，“操作结果”界面输出结果

3 得意之处

★ **FLTK 界面点击选择功能**，减少复制粘贴次数，十分酷炫！

```
265  /*=====*/
266  // STAR:选择图书
267  void MainWindow::selectBook(int row) {
268      auto& books: std::vector<Book> & = bookManager->getAllBooks();
269      if (row >= 0 && row < static_cast<int>(books.size())) {
270          const Book& book = books[row];
271          selectedISBN = book.getISBN();
272
273          isbnInput->value(book.getISBN().c_str());
274          titleInput->value(book.getTitle().c_str());
275          authorInput->value(book.getAuthor().c_str());
276          publisherInput->value(book.getPublisher().c_str());
277          stockInput->value(std::to_string(val: book.getStock()).c_str());
278          priceInput->value(std::to_string(val: book.getPrice()).c_str());
279      }
280  }
281  /*=====*/
```

★ **模糊查询**

利用 string 类成员函数 find() 实现了模糊查询功能，书名、作者、出版社都可以查询；并且每个函数都使用了 const 重载（如 findByISBN）支持只读访问，提升性能。

4 亟待完善之处

- GUI 界面和后端解耦程度不高，一些功能是在 mainWindow 类内进一步实现的，不够优雅！
- 尽管经过大量改进与迭代，但偶尔还是会出现内存泄露（段错误）。智能指针能够解决这个问题，但碍于期末复习缘故未能系统学习。亟需加学。

5 收获与体会

✧ is-A 还是 has-A?

is-A 是继承，has-A 是组合。继承与组合都可以用来表示上下级之间的关系，但是在这道题上却让我犯了难。

显而易见，本题所采用的几个类（BOOK，BOOKMANAGER，etc.）之间，并没有像平面与圆、父亲与儿子这样自然的层次关系。但是我经验尚浅，却急于想在课程设计里运用所学，这导致我在初始的 2 个版本中一门心思构造继承关系，试图找到虚继承接口，结果却导致代码晦涩难懂，难以下咽。慎重考虑下，我在 3.0 版本放弃了继承，转而使用友元函数进行权限管理。总之，工具在适当的地方才能发挥作用。

✧ 掌握了多种开发工具

我学会了使用 CMake 构建项目，用 Git 在不同设备同步进度，还掌握了 markdown 文档和 mermaid 的写法。

序号	知识点	说明
1	函数	本系统共定义了 91 个函数，包括带返回值、传入基本数据类型、指针、引用类型、构造函数、析构函数、void 类型函数等
2	容器	本项目使用了 vector 容器，用于存储对象的数据
3	文件	本项目使用 bin 文件做数据持久化存储
4	类与对象	定义了 6 个类，主要用于完成题目要求的操作和制作 GUI 窗口
5	安全头文件包含	#ifndef #define #endif
6	异常	定义了异常处理类，使用 try-catch 结构处理异常
7	友元函数	将运算符重载函数标记为友元函数，以便能够访问类的私有成员
8	运算符重载	重载了<<等运算符，便于设定格式输出数据
9	Lambda 表达式	隐式声明函数，简化代码，增强可读性

6 参考文献

[1] 柴田望洋. 明解 C++[M]. 管杰, 罗勇, 译. 北京: 人民邮电出版社, 2013.

[1] 洛谷网校. 深入浅出程序设计竞赛[M]. 北京: 电子工业出版社, 2020.